

# SimpleVLA-RL：通过强化学习扩展 VLA 训练

Haozhan Li<sup>\*,1</sup>, Yuxin Zuo<sup>\*,1</sup>, Jiale Yu<sup>\*,1</sup>, Yuhao Zhang<sup>\*,3</sup>, Zhaohui Yang<sup>3</sup>, Kaiyan Zhang<sup>1</sup>, Xuekai Zhu

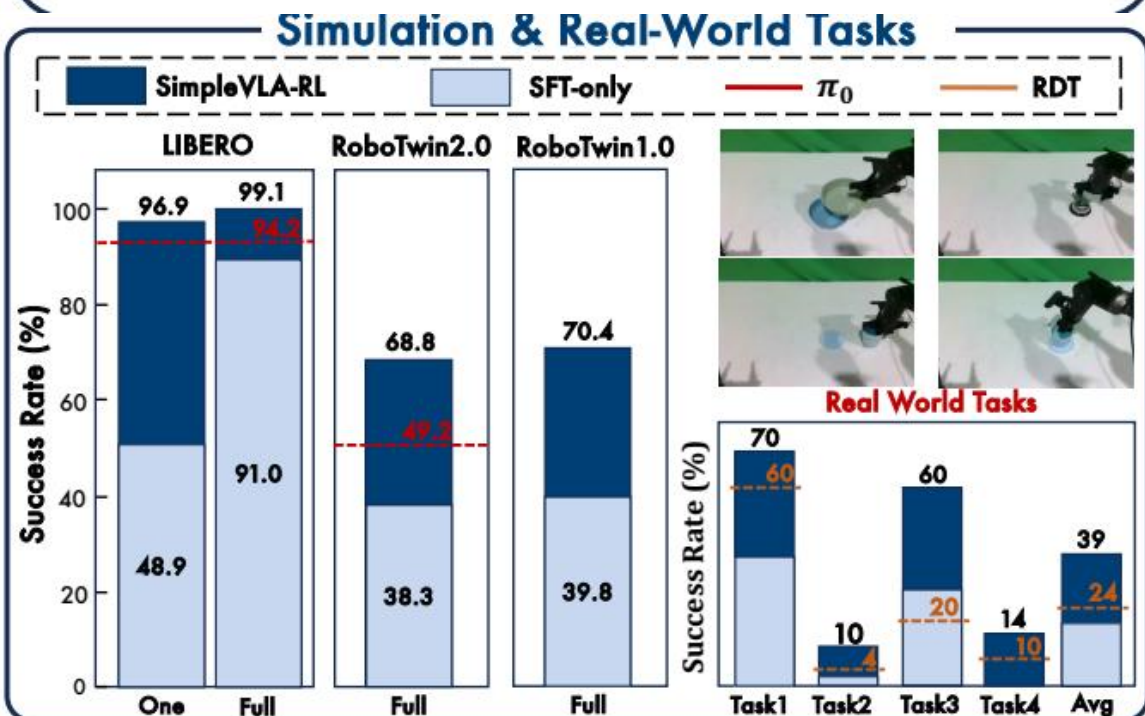
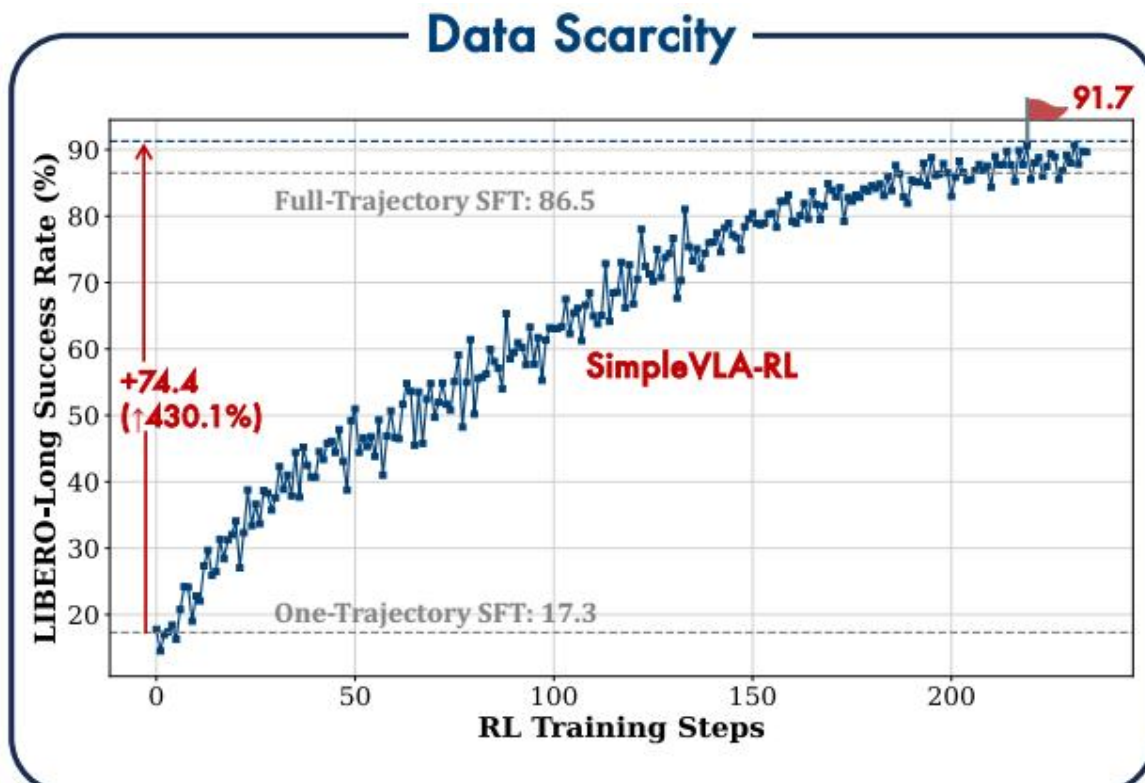
2026 年 8 月 17 日

## 摘要

Vision-Language-Action (VLA) 模型近年来已成为机器人操纵中的一种强大范式。尽管大规模预训练与监督微调 (SFT) 显著推动了该方向的发展，这类模型仍面临两个根本挑战：其一，支撑 SFT 规模化所需的大规模人工遥操作机器人轨迹既稀缺又昂贵；其二，在存在分布偏移的任务上，模型的泛化能力仍然有限。近期 Large Reasoning Models (LRMs) 的突破表明，强化学习 (RL) 能够显著提升逐步推理能力，这也自然引出一个问题：RL 是否也能提升 VLA 在长时程任务中的逐步动作规划能力？

本文提出 SimpleVLA-RL，这是一种面向 VLA 模型的高效 RL 框架。该方法基于 veRL，进一步引入了面向 VLA 的轨迹采样、可扩展并行化、多环境渲染以及优化后的损失计算。将其应用于 OpenVLA-OFT 时，SimpleVLA-RL 在 LIBERO 上取得了 SOTA 表现；结合本文提出的增强探索策略后，该方法在 RoboTwin 1.0 与 2.0 上甚至超过了  $\pi_0$ 。SimpleVLA-RL 不仅降低了对大规模数据的依赖、实现了稳健泛化，还在真实世界任务中显著优于 SFT。进一步地，我们在 RL 训练过程中发现了一种新的现象——“pushcut”：policy 会发掘出此前训练过程中从未出现过的新模式。

## 1 SimpleVLA-RL: 通过强化学习扩展 VLA 训练



### New Pattern Discovery: Pushcut Phenomenon

SFT Model



After SimpleVLA-RL



RL enables the policy to acquire novel actions never seen before.

### Generalization

## 1.1 目录

|                                        |    |
|----------------------------------------|----|
| 1 引言                                   | 3  |
| 2 预备知识                                 | 4  |
| 2.1 LLM 的强化学习建模                        | 4  |
| 2.2 VLA 的强化学习建模                        | 4  |
| 2.3 Group Relative Policy Optimization | 5  |
| 3 SimpleVLA-RL                         | 6  |
| 3.1 交互式 VLA rollout                    | 6  |
| 3.2 结果奖励建模                             | 7  |
| 3.3 探索增强                               | 7  |
| 3.4 训练目标                               | 9  |
| 4 实验                                   | 9  |
| 4.1 实验设置                               | 9  |
| 4.2 主要结果                               | 11 |
| 5 分析                                   | 12 |
| 5.1 克服数据稀缺问题                           | 12 |
| 5.2 泛化分析                               | 13 |
| 5.3 真实世界实验                             | 14 |
| 6 讨论                                   | 15 |
| 6.1 “Pushcut”：通过强化学习涌现的新模式             | 15 |
| 6.2 SimpleVLA-RL 的失效模式                 | 16 |
| 7 相关工作                                 | 17 |
| 7.1 面向大语言模型的强化学习                       | 17 |
| 7.2 Vision-Language-Action 模型          | 17 |
| 8 结论                                   | 18 |

## 2 1. 引言

Vision-Language-Action (VLA) 模型已成为让机器人在物理环境中完成多样且具有挑战性的操纵任务的一类有前景范式 [Firoozi et al., 2025]。这类模型展现出显著潜力, 但其研发过程本身也具有很高复杂度, 因为它需要在统一框架内同时整合视觉感知、语言理解与动作生成 [Kim et al., 2024; Zhong et al., 2025]。为此, 已有工作通常采用两阶段训练策略: 首先在大规模多模态数据上进行预训练 (pretraining), 数据来源包括人类操纵视频 [Sapkota et al., 2025]、图文对以及异构机器人数据集 [O’Neill et al., 2024]; 随后在高质量、大规模机器人轨迹数据上进行监督微调 (SFT), 以提升模型面向特定任务的能力。

这一范式已经取得了显著进展, 尤其是大规模预训练带来了明显收益。然而, 随着模型训练规模持续扩大, 特别是在下游任务优化中需要同步扩大 SFT 规模时, 若干关键挑战开始显现。1) **数据稀缺**: 扩大 SFT 规模需要更多由人工操作采集的机器人轨迹, 但这类数据既稀缺又成本高昂, 因而严重制约了可扩展性。机器人轨迹采集往往依赖精心设计的实验场景、多样化的操纵对象以及熟练操作员, 这使得数据规模与数据多样性都受到明显限制 [Mandlekar et al., 2021; Bu et al., 2025a;

Gao et al., 2024; Team et al., 2025a]。2) **泛化能力不足**：泛化仍是 VLA 的核心瓶颈之一，尤其是在组合式任务、长时程任务，或存在分布偏移的真实世界任务上更为突出。其根源在于，当前 VLA 的 SFT 通常依赖有限且与特定场景、特定任务强绑定的数据。因此，当 VLA 模型面对未见过的任务、环境或物体时，其性能往往不可避免地下降 [Liu et al., 2025a]。

近期 Large Reasoning Models (LRMs) 的进展，例如 DeepSeek-R1 [Guo et al., 2025a]，表明即便仅依赖结果奖励，强化学习 (RL) 也能推动模型能力取得显著提升。这些结果凸显了 RL 在增强模型逐步 chain-of-thought (CoT) 推理能力方面的重要作用 [Team et al., 2025b; Zeng et al., 2025; Yang et al., 2025]。这自然引出一个问题：RL 是否也能增强 VLA 模型逐步生成准确动作的能力？这一启发促使我们探索如何在 VLA 模型中复现 DeepSeek-R1 的成功经验，这也可能有助于缓解前述 SFT 面临的两项挑战。然而，将 RL 应用于 VLA 实际上还存在若干独特困难。传统机器人任务中的 RL 方法通常依赖手工设计的过程奖励，这严重限制了其可扩展性 [Ibarz et al., 2021; Kroemer et al., 2021; Ma et al., 2023]。此外，与 LLM 的 rollout 不同，VLA 需要与环境进行多轮交互，因此不仅速度更慢，而且成本也显著更高。

我们提出 SimpleVLA-RL，这是一种面向 VLA 模型的高效 RL 框架。该方法构建于面向 LLM 的通用 RL 框架 Volcano Engine Reinforcement Learning for LLMs (veRL) <sup>1</sup> [Sheng et al., 2024] 之上；通过实现面向 VLA 的交互式轨迹采样与损失计算，我们使 VLA 模型能够进行端到端、在线、基于规则的 RL。为进一步支持 VLA 的可扩展 RL，我们对 veRL 进行了扩展，引入并行多环境渲染以加速采样，并将其改造为一体化的训练-推理-渲染框架。令人惊讶的是，我们观察到 policy 在 RL 训练过程中发现了监督数据中从未出现过的新模式，其行为超出了已有监督数据所覆盖的范畴。我们将这一新现象称为“pushcut”。本文的主要贡献如下：

- **面向 VLA 的高效在线 RL 框架**：我们基于 veRL 构建了一个高效的端到端 VLA 在线 RL 框架，能够实现稳定且样本高效的训练，并针对渲染并行化以及分布式训练与推理进行了优化。
- **SOTA 性能**：我们引入了增强探索的策略，带来了稳定的 10–15% 性能提升。此外，SimpleVLA-RL 在 LIBERO 和 RoboTwin 1.0 & 2.0 上均超过了多个 SOTA baseline。

### 3 Introduction (第 2 部分)

- **数据效率与泛化能力**：在每个任务仅提供单条 demonstration 的条件下，RL 将 LIBERO-Long 上的成功率 (success rate) 从 17.1% 提升至 91.7%，并且在空间、物体和任务泛化方面显著优于 SFT。
- **真实世界部署能力**：在仿真中训练得到的 policy 能够有效迁移到真实环境，在不使用任何真实机器人数据的情况下，仍取得了显著的 sim-to-real 性能提升。

### 4 预备知识

为直观说明将面向 LLM 的强化学习 (RL) 方法扩展到 VLA 领域时仍存在的差距，本节分别形式化定义 LLM 与 VLA 模型中的 RL 问题，并给出它们的状态表征、动作空间、奖励函数以及环境。

## 4.1 LLM 的 RL 形式化

状态 ( $s_t$ ): 在第  $t$  步, 状态  $s_t$  由输入 prompt 与此前已生成的 token 构成:

$$s_t = (x_{\text{prompt}}, y_1, y_2, \dots, y_{t-1}), \quad (1)$$

其中,  $x_{\text{prompt}}$  表示初始 prompt,  $y_t$  表示第  $t$  个生成的 token。

动作 ( $a_t$ ): 动作对应于从词表  $\mathcal{V}$  中选择下一个 token。在每一步, policy 会输出关于各个 token 的概率分布, 并通过随机采样选出动作 token。形式化地,

$$a_t = y_t \in \mathcal{V}, \quad \text{where} \quad y_t \sim \pi_{\theta}(\cdot | s_t) = \text{softmax}(f_{\theta}(s_t)/T), \quad (2)$$

其中,  $f_{\theta}(s_t) \in \mathbb{R}^{|\mathcal{V}|}$  表示 LLM 输出的 logit,  $T$  为温度参数, 用于控制采样的随机性。

环境: 环境会在序列生成完成后提供奖励信号。在基于规则的设定中, 通常依据答案是否正确赋予二值奖励。另一类方法则使用学习得到的奖励模型, 或基于人类反馈的系统, 根据 helpfulness、harmlessness、任务对齐等标准给出连续奖励。奖励可写为:

$$r(\tau) = [\text{formula}], \quad \text{or} \quad r(\tau) = R_{\phi}(\tau) \in [0, 1], \quad (3)$$

其中,  $R_{\phi}$  是学习得到的奖励模型,  $\tau = (x_{\text{prompt}}, y_1, y_2, \dots, y_{T_{\text{seq}}})$  表示长度为  $T_{\text{seq}}$  的完整生成序列。

Rollout: 给定输入 prompt  $x_{\text{prompt}}$ , LLM 会从  $\pi_{\theta}(y_t | s_t)$  中逐步采样 token, 以自回归方式生成序列, 直到终止; 在此过程中, 不会接收环境提供的中间反馈。当温度参数  $T$  非零时, policy 能生成多样化的 rollout, 从而探索不同的解题路径。

## 4.2 RL Formulation for VLAs

状态 ( $s_t$ ): 状态由多模态观测构成, 包括视觉输入 (RGB 图像、深度图或点云)、proprioception 信息 (关节角、end-effector 位姿) 以及任务的语言指令。形式化地, 状态定义为:

$$\mathbf{s}_t = (o_t^{\text{vis}}, o_t^{\text{prop}}, l_{\text{task}}), \quad (4)$$

其中,  $o_t^{\text{vis}}$  表示多模态观测,  $o_t^{\text{prop}}$  表示 proprioception 信息,  $l_{\text{task}}$  表示语言指令。

动作 ( $a_t$ ): 动作是在机器人动作空间中的控制命令, 通常为 end-effector 增量或关节角目标, 其中  $a_t \in \mathbb{R}^d$  (e.g.,  $d = 7$ , 对应 6-DoF 位姿加 gripper 位置)。大多数 VLA policy 通过 diffusion-based action expert 或离散 action tokenizer 生成动作。动作定义如下:

$$a_t = \text{Decoder}(h_{\theta}(s_t)), \quad \text{Decoder} \in \{\text{Diffusion Expert, Action Tokenizer}\}, \quad a_t \in \mathbb{R}^d, \quad (5)$$

其中,  $h_{\theta}(s_t)$  表示 VLA 模型中状态  $s_t$  的隐状态,  $\text{Decoder}$  为动作解码器。

环境: 环境表示机器人所处的物理世界或仿真系统。它提供状态转移  $s_{t+1} = \text{Env}(s_t, a_t)$  以及奖励信号:

$$r_t = \alpha \cdot I_{\text{success}} + (1 - \alpha) \cdot \sum_i w_i \cdot \phi_i(s_t, a_t), \quad \alpha \in [0, 1], \quad I_{\text{success}} = \begin{cases} 1, & \text{if task success} \\ 0, & \text{otherwise} \end{cases}, \quad (6)$$



其中,  $\phi_i(s_t, a_t)$  表示过程奖励 (例如到目标的距离),  $w_i$  为权重,  $\alpha$  用于平衡结果奖励与过程奖励。

**Rollout:** VLA 模型通过与环境的迭代交互生成轨迹。在每个时间步, policy  $\pi_\theta$  以当前状态  $s_t$  为输入, 输出一个长度为  $k$  的 action chunk  $(a_t, a_{t+1}, \dots, a_{t+k-1})$ 。机器人按顺序执行这些动作, 环境再根据物理动力学生成更新后的状态。执行完成后, 模型以新状态  $s_{t+k}$  为输入, 生成下一个 action chunk。该过程持续进行, 直到任务完成或达到 episode 的最大长度, 最终通过交互式采样得到完整轨迹  $\tau = ((s_0, a_0), (s_1, a_1), \dots, (s_T, a_T))$ 。

### 4.3 2.3. Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) [?] 是一种强化学习 (RL) 方法。它通过组内相对归一化来计算 advantage, 从而不再显式依赖 value function。给定初始状态  $s_0$ , 行为 policy  $\pi_{\theta_{\text{old}}}$  生成  $G$  条轨迹  $\{\tau_i\}_{i=1}^G$ 。GRPO 的目标函数采用类似 PPO 的 clipping 机制, 并结合 KL regularization 约束 policy 更新:

$$[\text{formula}] \quad (7)$$

其中, 重要性采样比率  $r_{i,t}(\theta)$  与归一化 advantage  $\hat{A}_i$  定义为:

$$r_{i,t}(\theta) = \frac{\pi_\theta(a_{i,t}|s_{i,t})}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_{i,t})}, \quad \hat{A}_i = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \quad (8)$$

这里,  $R_i$  表示第  $i$  条轨迹的总回报,  $\epsilon > 0$  是 PPO 的 clipping 参数, 用于限制 policy ratio 的变化范围;  $\beta > 0$  是控制 KL regularization 强度的系数, 其参考对象为参考 policy  $\pi_{\text{refo}}$

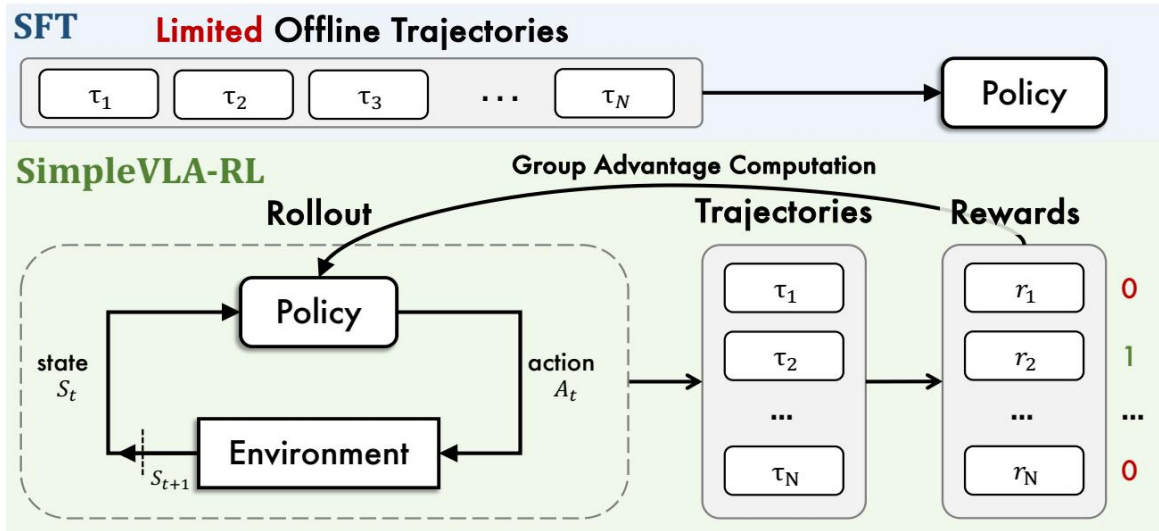


图 2: SimpleVLA-RL 的整体框架。

## 5 SimpleVLA-RL

DeepSeek-R1 [?] 通过采用简单且可扩展的基于规则的 reward 进行在线 RL, 取得了显著的性能提升, 展现出一种很有前景的训练范式。在本节中, 我们介绍 SimpleVLA-RL。如图 2 所示, 该方将这一基于规则的在线 RL 框架扩展到 VLA 模型, 以处理具身操纵任务。

具体而言，我们的训练框架包括以下步骤：首先，对每个输入通过随机采样生成多条 trajectory（见 §3.1）；随后，根据环境反馈为每条 trajectory 赋予一个简单的结果 reward（成功记为 1，失败记为 0；见 §3.2）；最后，结合这些 reward 及其对应的 action token 概率，计算 GRPO loss，并据此更新 policy 模型（见 §3.4）。

### 5.1 3.1. Interactive VLA Rollout

在轨迹生成方式上，针对 VLA 模型开展 RL 与 LLM 存在本质差异。为了支持在线 RL，policy 模型需要针对同一输入生成多样化轨迹，从而实现有效探索。对于 LLM，这种多样性通常可以通过对文本 token 分布进行随机采样自然获得 [Renze, 2024; De Rosa and Papa, 2021]。然而，VLA 模型由于其 action 解码策略不同，面临着更为特殊的挑战。现有 VLA 模型通常采用以下三类策略：1) 类似 LLM 的 action token 分布生成 [Kim et al., 2024; Black et al., 2024]；2) 基于 latent state 的 diffusion 去噪 [Liu et al., 2024; Cheang et al., 2025]；3) 通过 MLP 进行确定性回归 [Kim et al., 2025]。在这三者中，基于 token 的方案与 PPO 类 RL 算法最为兼容，因为它天然提供了 action 分布，这对于随机采样和 policy gradient 计算都至关重要。因此，我们采用这一方案：由 VLA 模型输出 action token 的概率分布，并通过随机采样生成多样化轨迹。

此外，对于给定输入 query，LLM 的 rollout 通常以自回归方式逐步生成 token，直到遇到停止 token 或达到最大输出长度。相比之下，VLA 的 rollout 需要与环境持续交互，以动态更新视觉观测和机器人状态（详见第 2 节）。这种 closed-loop 交互是必需的，因为机器人执行的每个动作都会改变环境状态，而后续动作必须基于实时感知反馈进行条件化。我们在代码清单 1 中给出了 LLM 与 VLA rollout 算法伪代码的对比。

```
def rollout(policy, dataset, number_sample=8, max_steps=None):
    rollout_dataset = []
    for batch in dataset:
        batch = batch.repeat(number_sample)
```

## 6 LLM 通过随机采样生成多样化输出

```
outputs = policy.generate(batch, temperature=1.0)
rollout_dataset.append((batch, outputs))
```

## 7 并行环境初始化与交互

```
states = env_process_pool.submit(envs.setup) for t in range(max_steps):
```

## 8 VLA 通过 temperature 生成多样化轨迹

对 action token 进行采样：

```
actions = policy.generate(states, temperature=1.0)
for s, a in zip(envs, states, actions):
    ...
```

```

states, dones = env_process_pool.submit(envs.step, actions)
# Remove completed tasks
active = [(e, s) for e, s, d in zip(envs, states, dones) if not d]
if not active:
    break
envs, states = zip(*active)
return rollout_dataset

```

**代码清单 1** | 所采用的 veRL rollout 函数伪代码：从基于 LLM 的生成扩展到交互式 VLA 采样，并结合环境同步并行机制。

### 8.1 3.2 结果奖励建模

SimpleVLA-RL 在 RL 训练中采用了一种非常直接的二值奖励函数。不同于传统 RL 方法通常需要精心设计奖励函数 [Hadfield-Menell et al., 2017; Knox et al., 2023; Booth et al., 2023]，我们遵循 DeepSeek-R1 的思路，仅依据任务是否完成，为整条轨迹赋予 0 或 1 的 trajectory-level reward。具体而言，当 VLA 模型成功完成任务时，整条轨迹的奖励记为 1；否则记为 0。在梯度计算时，这种轨迹级奖励会均匀传播到各个 action token。因此，成功轨迹中的所有 token 都被赋予奖励 1，而失败轨迹中的所有 token 都被赋予奖励 0。我们的奖励函数定义如下：

$$R(a_{i,t} | s_{i,t}) = [\text{formula}] \quad (9)$$

这种简单的结果级奖励虽然形式朴素，但效果有效：它具有良好的可扩展性，能够广泛适用于不同环境，同时避免了复杂的过程级设计 [Wu et al., 2021]。由于只关注任务是否完成，它也规避了任务特定奖励通常存在的不可迁移问题。

### 8.2 3.3 探索增强（第 1 部分）

以往工作 [Yu et al., 2025; Liu et al., 2025b,c; An et al., 2025] 已表明，在 RL 过程中鼓励探索至关重要。我们进一步观察到，这一点在 VLA RL 中更为关键。操纵任务通常存在较大范围的可行解；然而，VLA 模型往往会收敛到一组较为狭窄的解模式，这主要源于其训练轨迹的同质性，从而限制了 RL 的效率。促进探索能够鼓励模型发现新的策略并拓展解空间，这一特性在成功率（success rate）较低的场景中尤其有利。基于这一认识，我们引入了三项关键修改以增强 RL 训练中的探索性：1) 在轨迹 rollout 期间采用动态采样（dynamic sampling）；2) 调整 GRPO 训练目标中的 clip range；3) 在 rollout 时提高采样温度。

**动态采样（Dynamic Sampling）** 无 critic 的 RL 算法在轨迹被赋予相同奖励时会遭遇梯度消失问题。以 GRPO 为例，它通过组内相对归一化来计算 advantage，即将每个响应的奖励与其所属采样组内奖励的均值和标准差进行比较。当所有轨迹的奖励完全相同时，advantage 估计会变为零，进而产生零梯度，导致训练动态不稳定。

为了解决这一问题，我们采用动态采样（Dynamic Sampling）[Yu et al., 2025; Cui et al., 2025a]。该方法已在 LLM RL 中被证明是有效的 [Team et al., 2025b; Yu et al., 2025; Cui et al., 2025a; Shi et al., 2025]。在 rollout 过程中，我们剔除那些组内所有轨迹要么全部成功、要么全部失败的样本组。采样会持续进行，直到整个 batch 仅由成功与失败结果混合的样本组构成，其形式化表达如下：



$$0 < |\{\text{traj}_i(a_i, s_i) \mid \text{is\_successful}[\text{traj}_i(a_i, s_i)]\}| < G. \quad (10)$$

这一策略保证了 advantage 估计非零，并在整个训练过程中维持稳定的梯度传播。

**提高 clipping 上界 (Clipping Higher)** PPO 和 GRPO 都会对 importance sampling ratio 进行 clipping，以约束 trust region [Schulman et al., 2015]，并提升 RL 的稳定性 [Shao et al., 2024; Schulman et al., 2017]。然而，clipping 的上界会限制低概率 token 的概率增幅，因此可能抑制探索。遵循 DAPO [Yu et al., 2025] 的做法，我们将 GRPO 训练目标中的 clipping 区间由  $[0.8, 1.2]$  调整为  $[0.8, 1.28]$ 。

**更高的 rollout 温度 (Higher Rollout Temperature)** 近期关于 LLM RL 的研究表明，通过调整 rollout 温度来促进探索是广泛有效的，尤其是在更高温度下采样时，性能提升更为显著 [Liu et al., 2025c; An et al., 2025; Liao et al., 2025]。为了鼓励 VLA 模型在 rollout 阶段生成更加多样化的轨迹，我们将采样温度从 1.0 提高到 1.6。如图 3 所示，这些修改带来了显著的性能提升。



图 3: 三项关键探索增强策略的有效性：动态采样、更高的 rollout 温度，以及提高 clipping 上界。

### 8.3 3.3. 探索增强（第 2 部分）

表 1: RoboTwin 2.0 基于规划 horizon 与所需步数的任务分类

| 任务名称                             | 步数 (Steps)         | Horizon    | Horizon 分组           |
|----------------------------------|--------------------|------------|----------------------|
| <b>短 horizon 任务 (112–130 步)</b>  |                    |            |                      |
| lift_pot                         | 112                | Short      | 平均: 121 步; 数量: 4 个任务 |
| beat_block                       | 113                | Short      | 平均: 121 步; 数量: 4 个任务 |
| hammerpick_dual                  | 127                | Short      | 平均: 121 步; 数量: 4 个任务 |
| bottlesplace_phone_stand         | 130                | Short      | 平均: 121 步; 数量: 4 个任务 |
| <b>中等 horizon 任务 (151–223 步)</b> |                    |            |                      |
| move_can                         | 151                | Medium     | 平均: 176 步; 数量: 4 个任务 |
| potplace_a2b_left                | 155                | Medium     | 平均: 176 步; 数量: 4 个任务 |
| place_empty_cup                  | 174                | Medium     | 平均: 176 步; 数量: 4 个任务 |
| handover_mic                     | 223                | Medium     | 平均: 176 步; 数量: 4 个任务 |
| <b>长 horizon 任务 (283–313 步)</b>  |                    |            |                      |
| handover_block                   | 283                | Long       | 平均: 298 步; 数量: 2 个任务 |
| stack_bowls_two                  | 313                | Long       | 平均: 298 步; 数量: 2 个任务 |
| <b>超长 horizon 任务 (466–637 步)</b> |                    |            |                      |
| blocks_rank_rgb                  | 466                | Extra-Long | 平均: 552 步; 数量: 2 个任务 |
| put_bottles_dustbin              | 637                | Extra-Long | 平均: 552 步; 数量: 2 个任务 |
| 总体统计                             | 共 12 个任务, 平均 256 步 |            |                      |

### 8.4 3.4. Training Objective

我们采用经调整后的 GRPO 算法 [Shao et al., 2024], 在 VLA 模型上进行在线 RL 训练, 具体改动见第 3.3 节。此外, 遵循 DAPO [Yu et al., 2025] 的做法, 我们去除了 KL divergence 正则项。这样一来, 训练时不再需要 reference model, 从而降低了显存占用并加快训练速度。另一方面, KL penalty 会限制 policy 相对于固定 reference 的偏离幅度, 这可能抑制对新行为的探索。因此, 我们通过如下目标函数优化 policy:

$$[\text{formula}] \quad (11)$$

其中,

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(a_{i,t} \mid s_{i,t})}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})}, \quad \hat{A}_i = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \quad (12)$$

## 9 4. Experiments

### 9.1 4.1. Experimental Setup

**基准测试** 我们在广泛使用的仿真基准 LIBERO [?], RoboTwin1.0 [?] 和 RoboTwin2.0 [?] 上评估 SimpleVLA-RL, 并在 RoboTwin2.0 任务上进一步开展真实世界实验。LIBERO 是一个面向终身学习的基准, 聚焦于语言引导的 manipulation 任务, 覆盖多样化的物体类型、任务设定与环境。它包含五个任务套件: LIBERO-Goal、LIBERO-Spatial、LIBERO-Object、LIBERO-Long (10 个任务, 每个任务含 50 条 expert demonstrations) 以及 LIBERO-90 (90 个任务, 用于大规模多任务评估)。

表 2: 不同 VLA 模型在 LIBERO 上的主要结果。

| Model          | Spatial | Object | Goal | Long  | Avg  |
|----------------|---------|--------|------|-------|------|
| Octo           | 78.9    | 85.7   | 84.6 | 51.1  | 75.1 |
| OpenVLA        | 84.7    | 88.4   | 79.2 | 53.7  | 76.5 |
| Nora           | 92.2    | 95.4   | 89.4 | 74.6  | 87.9 |
| $\pi_0$ + FAST | 96.4    | 96.8   | 88.6 | 60.2  | 85.5 |
| $\pi_0$        | 96.8    | 98.8   | 95.8 | 85.2  | 94.2 |
| UniVLA         | 96.5    | 96.8   | 95.6 | 92.0  | 95.2 |
| OpenVLA-OFT    | 91.6    | 95.3   | 90.6 | 86.5  | 91.0 |
| w/ ours        | 99.4    | 99.1   | 99.2 | 98.5  | 99.1 |
| $\Delta$       | +7.8    | +3.8   | +8.6 | +12.0 | +8.1 |

表 3: 不同 VLA 模型在 RoboTwin1.0 上的主要结果。

| Model       | Hammer Beat | Block Handover | Blocks Stack | Shoe Place | Avg   |
|-------------|-------------|----------------|--------------|------------|-------|
| DP          | 0.0         | 12.0           | 7.1          | 4.3        | 5.9   |
| DP3         | 64.7        | 84.3           | 24.0         | 59.3       | 58.1  |
| OpenVLA-OFT | 67.2        | 61.6           | 7.1          | 23.4       | 39.8  |
| w/ ours     | 92.6        | 89.6           | 40.2         | 59.3       | 70.4  |
| $\Delta$    | +25.4       | +28.0          | +33.1        | +35.9      | +30.6 |

我们使用每个任务在 50 个留出测试场景上的平均成功率 (Success Rate, SR) 作为性能指标。此外, RoboTwin1.0 [?] 与 RoboTwin2.0 [?] 是面向双臂 manipulation 的仿真基准。RoboTwin1.0 提供了 17 个双手任务, 但场景与物体多样性相对有限。RoboTwin2.0 则扩展到多种机器人 embodiment 上的 50 个任务, 包含 731 个物体实例, 并引入了全面的 domain randomization, 包括 clutter、光照、背景、桌面高度和语言指令, 从而提升了任务多样性以及 sim-to-real 迁移能力。

在 RoboTwin2.0 的训练与评估中, 我们采用 Agilex Piper 机械臂, 并使用经过 domain randomization 的任务设置; 每个任务在 100 个留出测试场景上进行评估。针对 RoboTwin2.0, 我们选取了 12 个任务, 并依据每个任务的平均步数将其划分为 4 个 horizon level, 以进行更全面的评测。表 1 汇总了各任务的具体步数, 以及 RoboTwin2.0 中不同 horizon level 对应的步数范围。

## 9.2 4.1 实验设置 (第二部分)

**Backbones** 我们将 SimpleVLA-RL 应用于 OpenVLA-OFT [?], 这是一个当前具有较高性能与推理效率的 SOTA 自回归 VLA 模型。OpenVLA-OFT 基于 OpenVLA [?] 构建, 以视觉 encoder 和 LLaMA2-7B [?] 作为 backbone, 并采用 action chunk 与并行解码设计, 因此特别适合在线 RL 场景——这类场景中模型推理会被频繁调用。

我们实现的 OpenVLA-OFT 与官方版本存在差异。为提升训练和推理效率, 我们仅使用单视角图像、语言指令以及机器人的 proprioceptive state 作为模型输入, 而官方模型还额外使用了腕部相机图像。此外, 在 LIBERO 实验中, 我们不将机器人的 proprioceptive state 作为模型输入。就模型结构而言, 我们仅保留并行解码与 action chunking 设计。我们使用 LLaMA2 的输出头生成 action token, 并采用 cross-entropy loss; 而官方模型使用 MLP 生成连续动作, 并采用  $L1$  回归损失。由于模型输入和架构均有差异, 我们无法直接使用官方 checkpoint。因此, 我们在官方代码库基础上进行了修改, 并使用与官方实现相同的数据集和超参数, 从头执行了 SFT。

表 4: 不同 VLA 模型在 RoboTwin2.0 上的主要结果, 按任务时域长度组织。

| 短时域任务 (100–130 steps) |          |                   |                   |                   |       |
|-----------------------|----------|-------------------|-------------------|-------------------|-------|
| Model                 | Lift Pot | Beat Hammer Block | Pick Dual Bottles | Place Phone Stand | Avg   |
| $\pi_0$               | 51.0     | 59.0              | 50.0              | 22.0              | 45.5  |
| RDT                   | 45.0     | 22.0              | 18.0              | 13.0              | 24.5  |
| OpenVLA-OFT           | 10.1     | 28.1              | 29.7              | 17.1              | 21.3  |
| w/ ours               | 64.1     | 87.5              | 68.3              | 39.6              | 64.9  |
| $\Delta$              | +54.0    | +59.4             | +38.6             | +22.5             | +43.6 |

表 5: 不同 VLA 模型在 RoboTwin2.0 中等时域任务上的结果。

| 中等时域任务 (150–230 steps) |              |                |                 |              |       |
|------------------------|--------------|----------------|-----------------|--------------|-------|
| Model                  | Move Can Pot | Place A2B Left | Place Empty Cup | Handover Mic | Avg   |
| $\pi_0$                | 41.0         | 38.0           | 60.0            | 96.0         | 58.8  |
| RDT                    | 33.0         | 21.0           | 42.0            | 95.0         | 47.8  |
| OpenVLA-OFT            | 28.1         | 37.5           | 77.3            | 45.3         | 47.1  |
| w/ ours                | 61.2         | 45.3           | 94.2            | 89.2         | 72.5  |
| $\Delta$               | +33.1        | +7.8           | +16.9           | +43.9        | +25.4 |

表 6: 不同 VLA 模型在 RoboTwin2.0 长时域与超长时域任务上的结果。

| 长时域 (280–320 steps) 与超长时域任务 (450–650 steps) |                |                 |                   |                     |       |
|---------------------------------------------|----------------|-----------------|-------------------|---------------------|-------|
| Model                                       | Handover Block | Stack Bowls Two | Blocks Rank Rgb   | Put Bottles Dustbin | Avg   |
| $\pi_0$                                     | 39.0           | 53.0            | 45.0              | 36.0                | 43.3  |
| RDT                                         | 26.0           | 42.0            | 17.0              | 26.0                | 27.8  |
| OpenVLA-OFT                                 | 33.1           | 40.6            | 70.2              | 42.2                | 46.5  |
| w/ ours                                     | 57.8           | 75.8            | 81.3              | 60.9                | 69.0  |
| $\Delta$                                    | +24.7          | +35.2           | +11.1             | +18.7               | +22.4 |
| Overall Avg                                 | RDT: 33.3      | $\pi_0$ : 49.2  | OpenVLA-OFT: 38.3 | w/ ours: 68.8       | +30.5 |

**Baselines** 我们将方法与近期较先进的 VLA 模型进行比较, 包括 UniVLA [?], RDT-1B [?],  $\pi_0$  [?],  $\pi_{\text{fast}}$  [?], Nora [?], Open-VLA [?], Octo [?], DP [?] 以及 DP3 [?].



### 9.3 4.1. Experimental Setup (第 3 部分)

**实现细节** 在训练基础设施方面, 我们使用  $8 \times$  NVIDIA A800 80GB 进行全参数训练。训练超参数设置如下: 学习率  $\eta = 5 \times 10^{-6}$ , 训练 batch size 为 64, sampling count 为 8, mini-batch size 为 128, clip ratio  $\varepsilon_{low} = 0.2$ 、 $\varepsilon_{high} = 0.28$ , 温度  $T = 1.6$ 。

在 action chunk 的设置上, LIBERO 中的 action chunk 数量为 8, 而 RoboTwin1.0&2.0 中为 25。模型总共配置了 256 个 action tokens。最大环境交互步数在 LIBERO 中设为 512; 在 RoboTwin1.0&2.0 中, 则根据不同任务设为 200、400 或 800。

在 RL 训练的 rollout 阶段, 我们采用随机采样; 在评估阶段, 则采用 greedy sampling。为保证结果的可复现性, 每个 benchmark 均测试 3 次。

### 9.4 4.2. Main Results

我们在三个基准上评估 SimpleVLA-RL: LIBERO、RoboTwin1.0 和 RoboTwin2.0。对于每个基准, 我们均采用两阶段训练范式: 先进行 SFT, 再进行 SimpleVLA-RL, 且两者都基于 OpenVLA-OFT。相比之下, baseline 模型仅使用 SFT 训练。

对于 LIBERO 的四个任务套件 (task suite), 我们先对每个任务套件使用全部 500 条 demonstration 进行 SFT, 然后在对应的 500 个仿真场景上执行 RL。对于 RoboTwin1.0, 我们在单任务设置下为每个任务使用 50 条 demonstration 进行 SFT, 随后在每个任务的 100 个仿真场景上执行 RL。对于 RoboTwin2.0, 我们为每个任务使用 1,000 条 demonstration 进行 SFT, 随后在每个任务的 1,000 个场景上执行 RL。

表 2、表 3 和表 4 分别给出了三个基准上的结果: LIBERO、RoboTwin1.0 和 RoboTwin2.0。在 LIBERO 的四个任务套件上, SimpleVLA-RL 将经 SFT 调优后的 OpenVLA-OFT 模型的平均成功率 (success rate) 从 91% 提升到 99%, 达到 SoTA 水平, 并超过当前所有 VLA 模型, 例如  $\square$  和 UniVLA。对于 LIBERO-Long 中的长时程任务, SimpleVLA-RL 取得了 98.5% 的成功率, 相比 baseline (86.5%) 提升 12%, 相比  $\square$  (85.2%) 提升 13.3%。这些结果表明, 即使 SFT 已经取得了较强性能, SimpleVLA-RL 仍能显著提升模型表现。

在 RoboTwin1.0 的四个双臂任务上, SimpleVLA-RL 相比微调 (fine-tuning) 后的 OpenVLA-OFT baseline 实现了 30.6% 的增益, 将性能从 39.8% 提升至 70.4%。在 RoboTwin2.0 的 12 个双臂任务上, SimpleVLA-RL 实现了 80% 的相对提升, 将性能从 38.3% 提高到 68.8%, 并超过了包括  $\square_0$  (49.2%) 和 RDT (33.3%) 在内的 SoTA 方法。值得注意的是, 即使在两个需要多轮双臂交互的 Extra-Long-Horizon 任务上, 例如 “Blocks Rank Rgb” 和 “Put Bottles Dustbin”, SimpleVLA-RL 也分别实现了 11.1 和 18.7 个百分点的提升。SimpleVLA-RL 在所有时程层级上都表现出稳定增益, 验证了 outcome-level reward 即便面对复杂长时程任务也依然有效。

总体来看, 这些结果表明, SimpleVLA-RL 能够在不同基准上稳定提升模型性能, 且无需额外的 demonstration 数据。

## 10 5. 分析

本节从数据、泛化以及真实世界任务三个方面, 分析 SimpleVLA-RL 在应对制约 VLA 模型进一步发展与规模化的三项关键挑战时所发挥的作用。以下总结若干关键结论:

## 10.1 要点总结

1. **数据:** SimpleVLA-RL 能显著降低对 demonstration 数据的依赖, 从而有效缓解限制 VLA 扩展的数据稀缺瓶颈 (§5.1)。
2. **泛化:** 相比 SFT, SimpleVLA-RL 在空间配置、物体类型和任务设定上表现出更强的泛化能力 (§5.2)。
3. **真实世界任务:** SimpleVLA-RL 展现出很强的 sim-to-real 迁移能力。大规模仿真训练能够显著提升真实世界中的表现, 这表明它为扩展真实世界 policy 提供了一条很有前景的路径 (§5.3)。

## 10.2 5.1. 缓解数据稀缺问题

为操纵任务开发基础型 VLA 模型, 需要大规模 demonstration 数据用于训练 [Black et al., 2024; Liu et al., 2024; Intelligence et al., 2025]。这一依赖数据规模扩展的范式, 已经在 NLP 领域得到验证 [Touvron et al., 2023; Hofmann et al., 2022; Achiam et al., 2023]。然而, 对于具身操纵任务而言, 高质量轨迹数据的采集依然昂贵且困难, 这已成为 VLA 模型发展的一个根本性瓶颈 [Zhong et al., 2025; Bi et al., 2025]。因此, 作者进一步考察: 在 demonstration 轨迹极其有限的条件下, SimpleVLA-RL 是否仍能有效提升 VLA 模型, 从而缓解这一限制。

**实验设置** 为模拟 demonstration 数据稀缺的场景, 作者仅使用每个任务 1 条 demonstration 数据对 OpenVLA-OFT 进行微调 (fine-tuning), 记为 **One-Trajectory SFT**。考虑到四个 LIBERO 任务套件中每个都包含 10 个不同任务, 因此每个任务套件实际只使用 10 条 demonstration 数据。作为对比, 作者还进行了使用每个任务全部可用 demonstration 数据的实验, 即每个任务套件使用 500 条数据, 记为 **Full-Trajectory SFT**。在 One-Trajectory SFT 和 Full-Trajectory SFT 之后, 作者都进一步在对应的 SFT 模型上应用 SimpleVLA-RL。

**实验结果** 如表 7 所示, 当 demonstration 数据受限时, 单纯依赖 SFT 的局限性十分明显。与 Full-Trajectory SFT 设置下的结果相比, 在 One-Trajectory SFT 下, SFT 在 LIBERO-Spatial、LIBERO-Object 和 LIBERO-Goal 上的成功率 (success rate) 均不超过 63.6%, 而在长时程任务集 LIBERO-Long 上的性能更是下降到仅有 17.3%, 这凸显了仅靠 SFT 处理长时程任务的困难。

值得注意的是, 在 One-Trajectory SFT 的基础上加入 SimpleVLA-RL 后, 四个 LIBERO 任务套件上的平均成功率从 48.9% 大幅提升到 96.9%, 甚至超过了 Full-Trajectory SFT 的 91.0%。其中, LIBERO-Long 从 17.3% 提升到 91.7%; 其余三个任务套件的成功率也都超过了 98%, 并且均高于各自的 Full-Trajectory SFT 表现。进一步看, One-Trajectory SFT + RL (96.9%) 与 Full-Trajectory SFT + RL (99.1%) 之间的性能差距仅为 2.2%。

这些结果表明, 在数据稀缺场景下, SimpleVLA-RL 仍能显著提升 VLA 模型性能。这说明, 即便只有极少量 demonstration 数据, 借助带有结果反馈的试错式探索, SimpleVLA-RL 这类在线 RL 方法依然能够推动 VLA 模型训练进一步扩展。

表 7: One-Trajectory SFT 与 Full-Trajectory SFT 在 LIBERO 上的对比。

| Model               | LIBERO  |        |       |       |       |
|---------------------|---------|--------|-------|-------|-------|
|                     | Spatial | Object | Goal  | Long  | Avg   |
| One-Trajectory SFT  |         |        |       |       |       |
| OpenVLA-OFT         | 63.6    | 54.9   | 59.6  | 17.3  | 48.9  |
| w/ ours             | 98.2    | 98.7   | 98.8  | 91.7  | 96.9  |
| $\Delta$            | +34.6   | +43.8  | +39.2 | +74.4 | +48.0 |
| Full-Trajectory SFT |         |        |       |       |       |
| OpenVLA-OFT         | 91.6    | 95.3   | 90.6  | 86.5  | 91.0  |
| w/ ours             | 99.4    | 99.1   | 99.2  | 98.5  | 99.1  |
| $\Delta$            | +7.8    | +3.8   | +8.6  | +12.0 | +8.1  |

### 10.3 5.2. 泛化分析

VLA 模型的泛化能力仍然是一个关键挑战 [Zhong et al., 2025; Liu et al., 2025a; Intelligence et al., 2025]。本小节从三个维度评估 SFT 与在线 RL 方法（如 SimpleVLA-RL）对 VLA 泛化能力的影响：空间泛化（LIBERO-Spatial）、对象泛化（LIBERO-Object）以及任务泛化（LIBERO-Goal）。

**设置** 我们在三个任务套件上开展实验：LIBERO-Spatial、LIBERO-Object 和 LIBERO-Goal。每个任务套件均包含 10 个不同任务。对于每个 LIBERO 任务套件，我们随机选取其中 9 个任务作为已见任务，用于 RL 或 SFT 训练；剩余 1 个任务作为未见任务，用于分布外评估。对两种方法而言，我们都首先在 One-Trajectory SFT 设定下对 OpenVLA-OFT 进行微调（fine-tuning），以获得一个成功率非零的基础模型；这是因为原始模型在 LIBERO 上的成功率仅为 0%，无法进行在线 RL。对于 SFT，我们进一步使用每个任务套件中 9 个已见任务的全部 450 条 demonstration 轨迹，对 One-Trajectory SFT 基础模型 (§5.1) 继续微调。此外，我们也在这一 One-Trajectory SFT 基础模型上执行 SimpleVLA-RL。我们绘制了这样一条关系曲线：随着训练过程中模型在训练任务上的成功率不断提升，其在未见任务上的表现如何变化。

**结果** 结果如图 4 所示。在训练过程中，SFT 和 RL 在训练任务上都能达到超过 90% 的成功率。然而，它们在未见任务上的表现出现了显著分化。随着训练期间已见任务成功率的提高，SimpleVLA-RL 在

不同设定下都能在未见任务上持续提升性能。相比之下，SFT 存在严重的过拟合问题。尽管它在训练任务上取得了较高性能，但在大多数未见任务上性能反而下降，且常常出现灾难性遗忘，成功率降至 0%。这表明，RL 训练能够使 VLA 模型在学习来自多样任务的可泛化技能的同时，保留先前已获得的能力。

在 LIBERO-Goal 的三个未见任务上，SFT 的成功率在训练初期便立即降为 0%。这可能是因为 LIBERO-Goal 任务涉及多样化的对象与操纵策略，任务之间可迁移的共享成分较少；而 SFT 的泛化更依赖于任务分布之间的相似性。与之相对，我们的 SimpleVLA-RL 不仅避免了性能退化，还在这三个未见任务上分别取得了 5%–15% 的提升。

在 LIBERO-Object 上，SFT 仅在 Unseen Task 3 上从 57.8% 提升到 74.6%，但在另外两个任务上均未奏效。相比之下，我们的方法在全部三个未见任务上都有提升，其中在 Unseen Task 2 上提升了 36.5%，在 Unseen Task 3 上提升了 16.4%。

在 LIBERO-Spatial 上，我们观察到一致的趋势：SFT 在 Unseen Task 1 上下降了 10%，并在其余任务上完全失败。相反，我们的方法将 Unseen Task 1 的表现从 43.3% 提升到 71.8%，并在另外两个任务上分别取得了 7.1% 和 13.3% 的增益。总体而言，SimpleVLA-RL 在提升 VLA 模型泛化能力方面始终优于 SFT，即便是在最具挑战性的跨任务场景中也是如此。

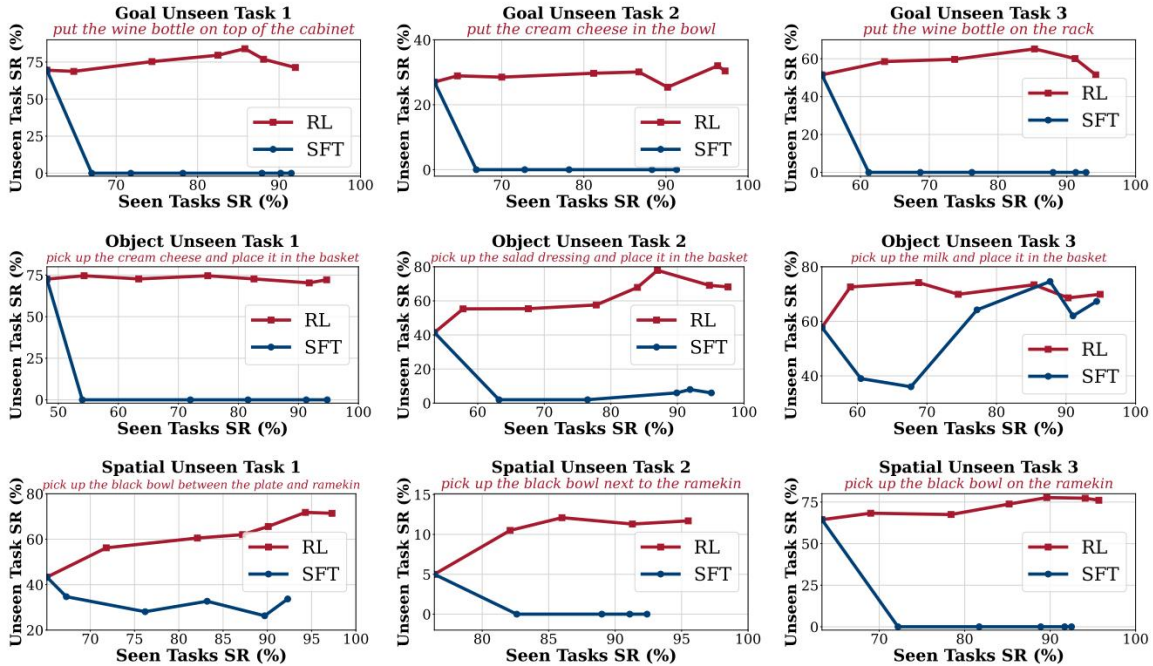


图 4: LIBERO 上的泛化分析: Goal Unseen (上)、Object Unseen (中)、Spatial Unseen (下)。

表 8: 真实世界实验 (sim2real) 结果。

|             | Stack Bowls | Place Empty Cup | Pick Bottle | Click Bell | Avg   |
|-------------|-------------|-----------------|-------------|------------|-------|
| RDT         | 60.0        | 4.0             | 10.0        | 20.0       | 23.5  |
| OpenVLA-OFT | 38.0        | 2.0             | 0.0         | 30.0       | 17.5  |
| w/ ours     | 70.0        | 10.0            | 14.0        | 60.0       | 38.5  |
| $\Delta$    | +32.0       | +8.0            | +14.0       | +30.0      | +21.0 |



### 10.4 5.3 真实世界实验

为评估 SimpleVLA-RL 在真实世界中的有效性，我们在四个 RoboTwin2.0 任务<sup>2</sup> 上开展了 sim-to-real 实验：Stack Bowls、Handover Block、Pick Bottle 和 Click Bell。我们采用 OpenVLA-OFT 作为 policy 模型，RDT 作为 baseline 模型，并在两台 AgileX Piper 机械臂上执行实验。对于每个任务，我们首先使用 1000 条仿真轨迹进行 SFT；随后，在 SFT 模型基础上，利用 1000 个仿真场景应用 SimpleVLA-RL，得到 RL 模型。整个训练过程仅使用仿真数据，不包含任何真实世界 demonstration。我们在真实环境中的干净桌面、未见背景条件下，对 SFT 模型和 RL 模型进行评测。每个任务测试 50 次。作为对比，RDT baseline 模型仅经过 SFT 阶段训练。

表 6 中的 sim-to-real 结果表明，SimpleVLA-RL 能够显著提升 VLA 模型在真实世界中的成功率 (success rate)：平均成功率由 17.5% 提升至 38.5%，超过了 RDT 的 23.5%。例如，在 Stack Bowls 任务上，SimpleVLA-RL 带来了 96% 的相对提升，将性能从 32% 提高到 70%，并优于 RDT 的 60%。在 Pick Bottle 任务中，由于该任务对动作精度要求更高，如果机械臂在第一次尝试时未能精确对齐，瓶子就会倾倒，因此 SFT 模型完全失败，而 SimpleVLA-RL 仍取得了 15% 的成功率，说明其在提升动作精度方面是有效的。

总体来看，借助 SimpleVLA-RL 在仿真中进行低成本、大规模、高并行的 RL 训练，作者显著提升了基于仿真训练的 VLA 模型在真实世界中的表现。这表明了一条很有前景的真实策略扩展路径：利用丰富的仿真资产和高保真模拟器，开展高性价比的 RL 训练，从而在真实部署中获得更优性能。

### 10.5 6.1. “Pushcut”：通过 RL 涌现的新模式

任务：Move Can Pot

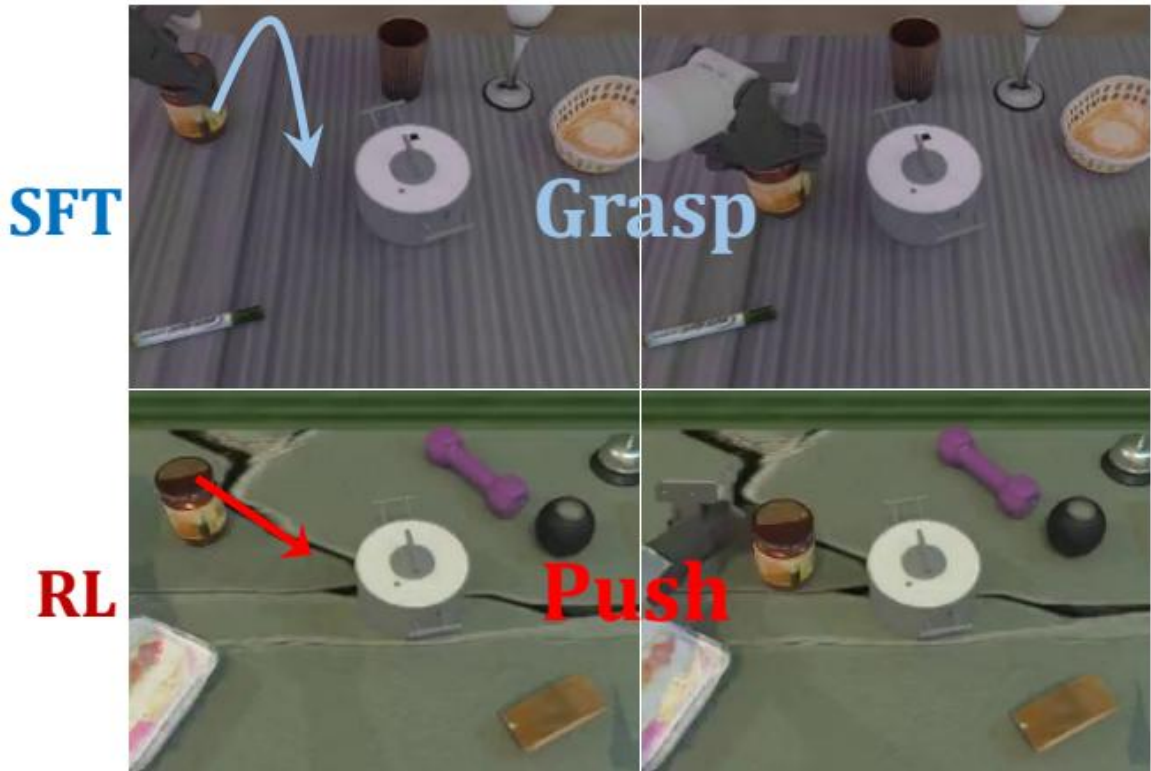


图 5: “move can pot” 任务示意：模型学会了通过推送易拉罐完成任务，而不是像 demonstration 数据中那样采用抓取-移动-放置。

任务：Place A2B Right

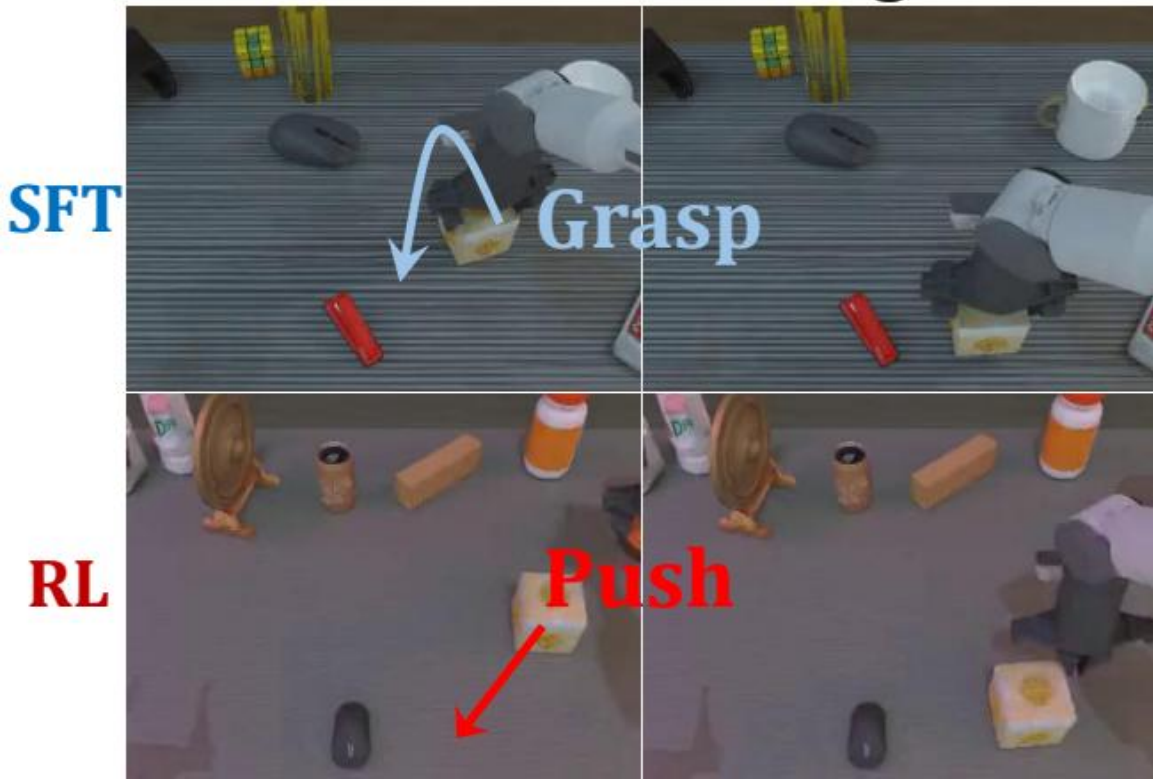


图 6: “place a2b right” 任务示意：模型学会了将物体 A 直接推到目标位置，而不是像 demonstration 数据中那样先抓取再移动并放置。

图 5 展示了我们在 RL 训练过程中观察到的一种涌现行为：“pushcut”。在 SimpleVLA-RL 的训练阶段，我们发现 VLA 模型能够学会 demonstration 数据中未出现过的新行为模式。

具体而言，在 RoboTwin2.0 的 move can pot 任务中，目标是将易拉罐移动到指定 pot 邻近的位置。所有 demonstration 轨迹都一致采用“抓取-移动-放置”策略，如图 5(a) 上半部分所示。

值得注意的是，经过 RL 训练后，VLA 模型自主发现了一种更高效的解法：它不再抓取易拉罐，而是直接将其推到目标位置，如图 5(a) 下半部分所示。我们将这一涌现现象称为“pushcut”，即一种由推送驱动的 shortcut，它体现了模型绕过传统“抓取-移动-放置”流程的能力。类似的涌现行为也出现在 place a2b left/right 任务中。对于该任务，demonstration 数据通过抓取物体 A 并将其放置在物体 B 旁边来完成任务；而经过 RL 训练的模型则学会了仅通过推送物体 A 到达目标位置来完成任务，如图 5(b) 所示。“pushcut”现象与 DeepSeek-R1 [?] 中观察到的“Aha Moment”高度相似：二者都是在 RL 过程中涌现出的新模式，即智能体在训练中自行发掘出新的有效策略。

这一现象凸显了 SFT 与 RL 之间的根本差异。SFT 只能复现 demonstration 中已有的固定模式，而 RL 则通过 reward 驱动的探索发掘新策略。在 RL 训练过程中，有效行为会因正 reward 而被强化，效率较低的行为则会逐步被淘汰。结果级 reward 进一步促进了这类涌现策略的出现：只要任务成功完成，无论采用抓取还是推送，都能获得等价的 reward。由于这种稀疏 reward 设计避免了过程级监督带来的程序约束，智能体因而拥有更大的探索空间，并能够发现那些事先未预料到但同样有效的解法。



## 10.6 6.2. SimpleVLA-RL 的失败模式

表 7 展示了初始模型能力对 SimpleVLA-RL 性能的影响。

表 9: 初始模型能力对 SimpleVLA-RL 性能的影响

|                          | RoboTwin2.0  |                |                 |                   |                   | Avg   |
|--------------------------|--------------|----------------|-----------------|-------------------|-------------------|-------|
|                          | Move Can Pot | Place A2B Lift | Place A2B Right | Place Phone Stand | Pick Dual Bottles |       |
| 0 trajs SFT +RL (训练前)    | 0            | 0              | 0               | 0                 | 0                 | 0     |
| 0 trajs SFT +RL (训练后)    | 0            | 0              | 0               | 0                 | 0                 | 0     |
| 100 trajs SFT +RL (训练前)  | 9.4          | 7.8            | 7.8             | 10.1              | 1.2               | 7.3   |
| 100 trajs SFT +RL (训练后)  | 51.6         | 25.0           | 27.2            | 18.8              | 4.3               | 25.4  |
| $\Delta$                 | +42.2        | +17.2          | +19.4           | +8.7              | +3.1              | +18.1 |
| 1000 trajs SFT +RL (训练前) | 28.1         | 37.5           | 28.7            | 17.1              | 29.7              | 28.2  |
| 1000 trajs SFT +RL (训练后) | 61.2         | 45.3           | 37.5            | 39.6              | 68.3              | 50.4  |
| $\Delta$                 | +33.1        | +7.8           | +8.8            | +22.5             | +38.6             | +22.2 |

本小节分析 SimpleVLA-RL 在何种条件下会失效,以及影响其效果的关键因素。我们在 RoboTwin2.0 的五个任务上进行实验,发现模型先验是决定 RL 是否有效的核心因素。

**实验设置** 每个任务都在单任务设定下、采用 domain randomization 进行训练。我们比较三种模型变体:

1. 不经过轨迹微调的 OpenVLA-OFT 基础模型 (0 trajectories SFT);
2. 每个任务使用 100 条 demonstration 轨迹进行微调的模型 (100 trajectories SFT);
3. 每个任务使用 1000 条 demonstration 轨迹进行微调的模型 (1000 trajectories SFT)。

所有模型都在 1000 个训练场景上接受 SimpleVLA-RL 训练,并在 100 个留出的测试场景上进行评估。

**当基础模型完全不具备初始任务能力时,RL 会彻底失效** 表 7 给出了相关结果。基础模型(0-trajectory SFT)在所有任务上的成功率 (success rate) 均为 0%,没有表现出任何与任务相关的行为。尽管经过了大规模预训练,OpenVLA 的 zero-shot 泛化能力依然极其有限,这一点与 Kim et al. [2025] 的结论一致。由于采样过程中无法生成任何成功轨迹,且训练中只使用结果奖励 (outcome rewards),不使用过程奖励 (process rewards),因此所有轨迹获得的奖励都为 0。结果是,RL 无法带来任何性能提升,模型表现始终维持在 0%。

**模型先验会显著影响 RL 的有效性** 初始能力与 RL 后的性能强相关。100-trajectory SFT 模型的平均成功率从 7.3% 提升到 25.4%,增幅为 18.1%;1000-trajectory SFT 模型则从 28.2% 提升到 50.4%,增幅为 22.2%。这一趋势在各个任务上都一致存在。以 move can pot 任务为例,100-trajectory SFT 模型从 9.4% 提升到 51.6%,而 1000-trajectory SFT 模型从 28.1% 提升到 61.2%。这些结果表明,更强的初始能力能够为探索提供更有效的起点,从而带来更大的性能提升。

**RL 的有效性存在阈值：初始能力过低时，提升几乎可以忽略** 我们的结果进一步表明，RL 的有效性受一个性能阈值约束。当初始成功率很低时，仅依赖结果奖励的在线 RL 只能带来极为有限的改进。例如，在 pick dual bottles 任务中，100-trajectory SFT 模型仅从 1.2% 提升到 4.3%，而 1000-trajectory SFT 模型则从 29.7% 提升到 68.3%。类似地，在 place phone 任务中，100-trajectory SFT 模型的增幅为 8.7%，而 1000-trajectory SFT 模型的增幅达到 22.5%。这些结果说明，要让 RL 真正发挥作用，模型必须先具备最低限度的任务胜任能力。低于这一阈值时，探索基本无效，RL 也难以产生有意义的收益。

## 11 相关工作

### 11.1 7.1. 用于大语言模型的强化学习

用于大语言模型 (LLMs) 的强化学习 (Reinforcement Learning, RL) 近年来取得了显著成功，展现出诱导复杂推理行为的能力，例如自验证 (self-verification) 与迭代优化 (iterative optimization)，从而显著提升模型在推理任务上的表现 [Guo et al., 2025a; Zeng et al., 2025; Cui et al., 2025a; Jaech et al., 2024; Liu et al., 2025d; Zuo et al., 2025]。近期大型推理模型 (LRMs) 的进展，例如 DeepSeek-R1 [Guo et al., 2025a]，进一步表明：即便仅使用简单的基于规则的奖励，RL 依然能够有效增强推理能力，GRPO [Shao et al., 2024] 就是一个典型例子。这一路线与基于人类反馈的强化学习 (RLHF) [Ouyang et al., 2022] 有显著区别。后者通常使用 Proximal Policy Optimization (PPO) [Schulman et al., 2017] 等算法，将基础模型对齐到人类偏好，并且高度依赖偏好建模。

近期研究越来越关注如何增强强化学习中的探索能力，以支持更长的训练时程并进一步提升性能。DAPO [Yu et al., 2025] 提出了 Clip-Higher，这是一种将 PPO clipping 解耦的变体，它为上界设置了相对于下界更高的阈值 (e.g.,  $\epsilon_{\text{low}} = 0.2$ ,  $\epsilon_{\text{high}} = 0.28$ )。这一调整使得原本似然较低、但可能具有价值的 token 概率能够上升，从而鼓励探索。在此基础上，POLARIS [An et al., 2025] 采用分阶段的温度课程 (staged curriculum)，逐步提高温度 (例如，对 7B 模型使用 0.7 1.0 1.1)，以渐进式扩展轨迹多样性，并促进更稳健的 policy 发现。

与此同时，Entropy Mechanism [Cui et al., 2025b] 针对长程训练中持续存在的熵塌缩 (entropy collapse) 问题，提出了 Clip-Cov 和 KL-Cov 等方法。其核心思想是选择性地裁剪概率，或对高协方差 token 施加惩罚，从而维持有效探索。类似地，ProRL [Liu et al., 2025b] 将 KL 控制与参考 policy 重置相结合，以在不损害性能的前提下保持训练稳定性并延长训练过程。

另一条互补的研究路线则通过温度调节来控制熵。Acereason-nemotron 1.1 [Liu et al., 2025c] 主张调整温度，使缩放后的熵稳定在某个目标值附近 (例如 0.3)，从而在探索与利用之间取得平衡。Liao et al. [2025] 则进一步提出了一种动态调度器，能够随时间自适应调整温度，以维持稳定熵，并由此支持持续的性能提升。

### 11.2 7.2 Vision Language Action Models

在机器人操纵任务中，VLA 模型 [Kim et al., 2024; Black et al., 2024; Liu et al., 2024; Kim et al., 2025; Bu et al., 2025b; Pertsch et al., 2025; Hung et al., 2025; Intelligence et al., 2025] 相比传统的基于 policy 的方法 [Ma et al., 2022; Yuan et al., 2024]，表现出更强的性能与任务泛化能力。这类模型通常通过统一的端到端训练，将 VLM 或 LLM backbone 与动作模块进行整合 [Zhong et al., 2025]。这种范式使模型能够同时具备全面的多模态理解能力与细粒度运动控制能力 [Firoozi et al., 2025]。

当前, 大量研究正围绕如何提升 VLA 模型的有效性展开。例如, E-COT [Zawalski et al., 2024; Chen et al., 2025b] 引入了 Embedded Chain of Thought (ECoT), 以增强 VLA 模型的空间推理能力。RDT-1B 和 VPP [Liu et al., 2024; Hu et al., 2024] 为 VLA 模型提出了基于 diffusion 的框架。Agibot world 和 Roboverse [Bu et al., 2025a; Geng et al., 2025] 致力于构建更大规模的仿真环境与轨迹数据集, 从而提升 VLA 模型的 sim-to-real 迁移能力与泛化能力。此外, Dexmimicgen [Jiang et al., 2024] 探索了自动化生成高质量轨迹数据的方法, 以缓解机器人领域中的数据稀缺问题。

尽管 VLA 方向发展迅速, 模仿学习仍然是 VLA 模型的主导训练范式 [Kim et al., 2024; Sapkota et al., 2025; Black et al., 2024; Liu et al., 2024; Kim et al., 2025; Bu et al., 2025b; Pertsch et al., 2025; Hung et al., 2025; Intelligence et al., 2025]。当前的 VLA 模型通常遵循两阶段范式: 首先在多模态数据上进行预训练 (pretraining), 例如 Open X-Embodiment [O’Neill et al., 2024]; 随后在采集到的机器人轨迹上进行 SFT。然而, 模仿学习受限于对昂贵且高质量轨迹数据的依赖, 同时对未见场景的泛化能力也较弱。

**VLA 中的强化学习方法** 近年来, 一些工作开始尝试将 RL 引入 VLA 训练。GRAPE [Zhang et al., 2024] 利用 Direct Preference Optimization (DPO) [Rafailov et al., 2023], 通过融合人类偏好来训练 VLA 模型。ConRFT [Chen et al., 2025c] 引入 Reinforced Fine-Tuning [Trung et al., 2024], 在真实环境中训练 VLA 模型, 并通过 RL 与 SFT 交替进行的多轮训练迭代优化 VLA。ReinboT [Zhang et al., 2025] 关注稠密奖励设计, 并通过奖励最大化来优化 VLA 模型。Guo et al. [2025b] 提出了一种结合 Supervised Fine-Tuning (SFT) 与 RL 阶段的迭代训练框架, 以缓解训练不稳定和计算开销过高的问题。

更近期的工作进一步推动了 VLA RL 方法的发展。我们的工作是最早系统探索 VLA 在线 RL 的研究之一, 并于 2025 年 5 月公开发布了代码<sup>3</sup>。与此同时, RIPT-VLA [Tan et al., 2025] 研究了一个高度相关的问题, 并采用 RLOO [Ahmadian et al., 2024] 进行 VLA 的 RL 训练。此外, Liu et al. [2025a] 研究了 RL 对 VLA 泛化能力的影响, 结果表明, 在未见环境、未见物体和未见纹理上, RL 相比 SFT 能带来显著提升。RLinf [Team, 2025] 设计了一个灵活且可扩展的 VLA RL 框架, 将渲染、推理与训练统一起来, 从而同时提升 VLA 的训练效率与性能。VLA-RL [Lu et al., 2025] 将 PPO 算法应用于 VLA 模型。TGRPO [Chen et al., 2025d] 使用 Claude3.7 对轨迹进行评估, 并基于 GRPO 优化 VLA。RFTF [Shu et al., 2025] 在具身场景中利用 value model 生成稠密奖励, 用于 VLA 在线 RL。

与上述工作相比, 我们的论文进一步探讨了 VLA RL 在真实机器人任务中的有效性。同时, 我们也对 VLA RL 如何缓解数据稀缺挑战、以及如何提升 policy 泛化能力, 进行了较为全面的分析。

## 12 8. Conclusion

本文提出了 SimpleVLA-RL, 一个面向 VLA 模型设计的 RL 框架。该方法在 veRL 的基础上, 进一步扩展了 VLA 特定的轨迹采样机制, 以及并行化的训练-推理-渲染能力, 从而支持可扩展且高样本效率的在线 RL。实验表明, SimpleVLA-RL 在数据效率、泛化能力和 sim-to-real 迁移方面均取得了显著提升。其在 LIBERO 和 RoboTwin 基准上的一致性能增益进一步说明, RL 不仅有望缓解 SFT 面临的数据稀缺问题, 还能够显著提升 VLA 模型的泛化能力。我们希望这些发现能够为构建更加自主、更加自适应的机器人模型提供新的路径。

## 13 参考文献

参考文献条目较多，此处保留原论文英文文献列表，请参阅原文 PDF 的 References 一节。